Attention Residuals

Softmax attention over depth replaces fixed residual sums

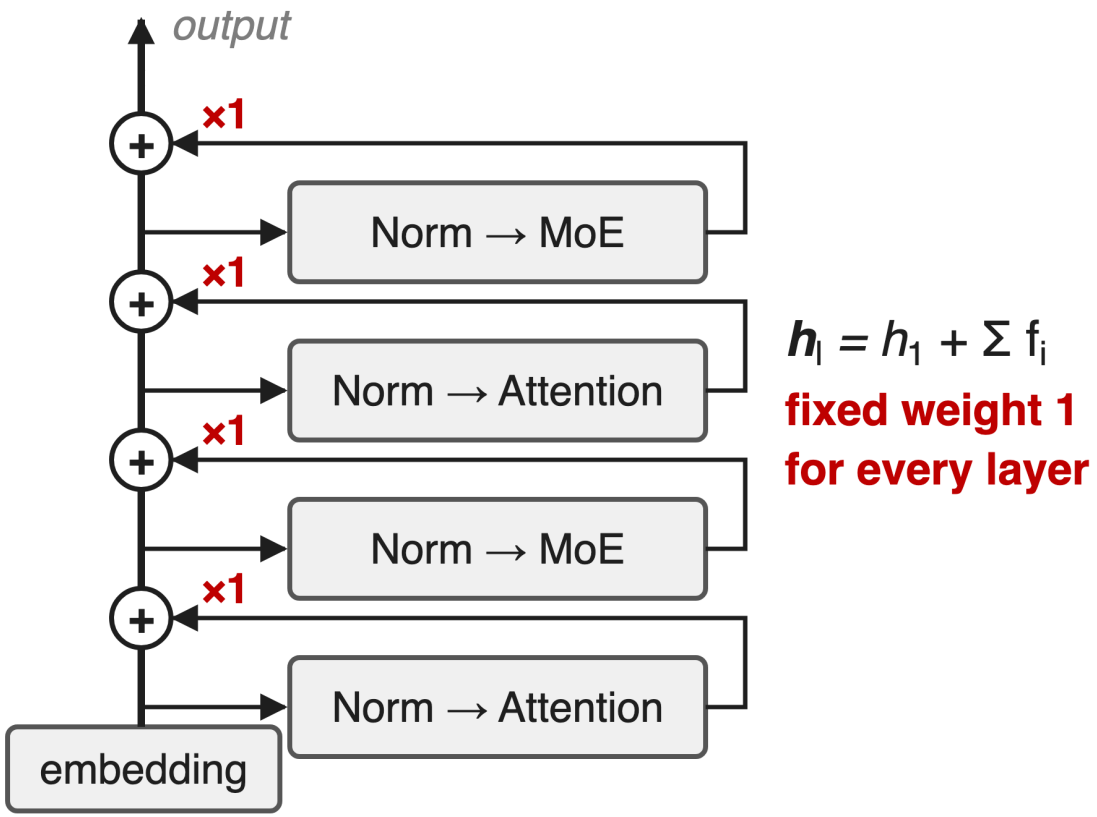
Moonshot AI · Kimi K3 (Attention Residuals report)

Explainer deck · Industrial Pretraining Architecture & Infra Tech Landscape (2026)

Model Shape & Depth Scaling · arxiv.org/abs/2603.15031

Every layer adds into one residual stream

- PreNorm LLMs: h ← h + f(Norm(h))
- Each output is added with weight **1**
- Kimi K3 replaces this in all **93** layers



Fixed residuals cost 1.25× compute

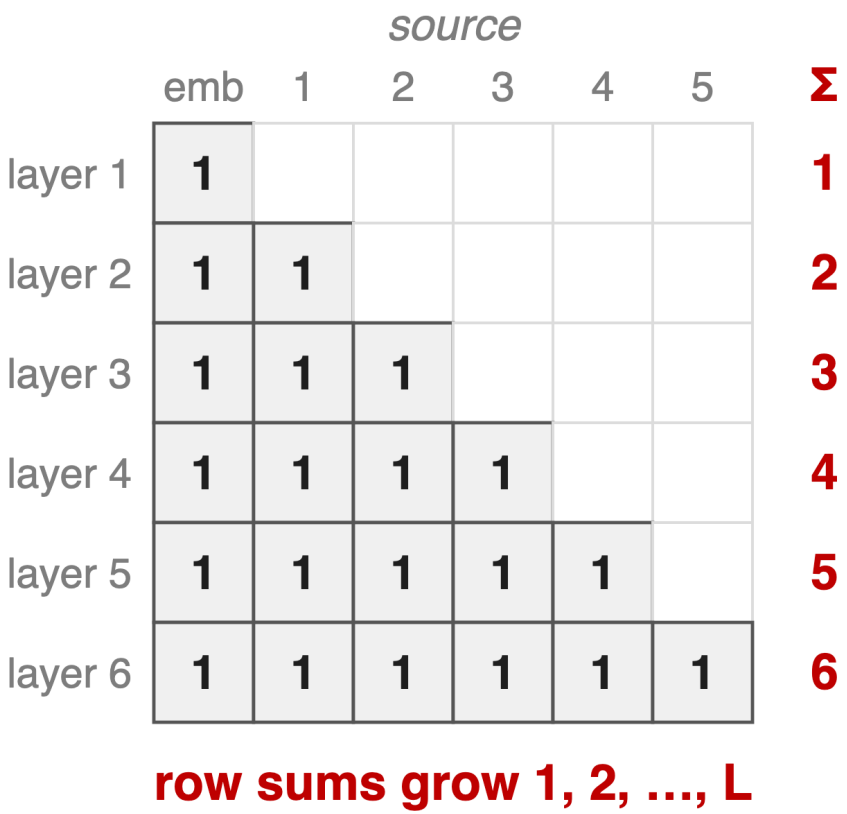
1.25×

compute a standard-residual (PreNorm) model needs to reach the validation loss of the same model with Block AttnRes

Attention Residuals, arXiv 2603.15031, §5.1 and Fig. 4 (scaling-law fit over five sizes, 194M–528M active)

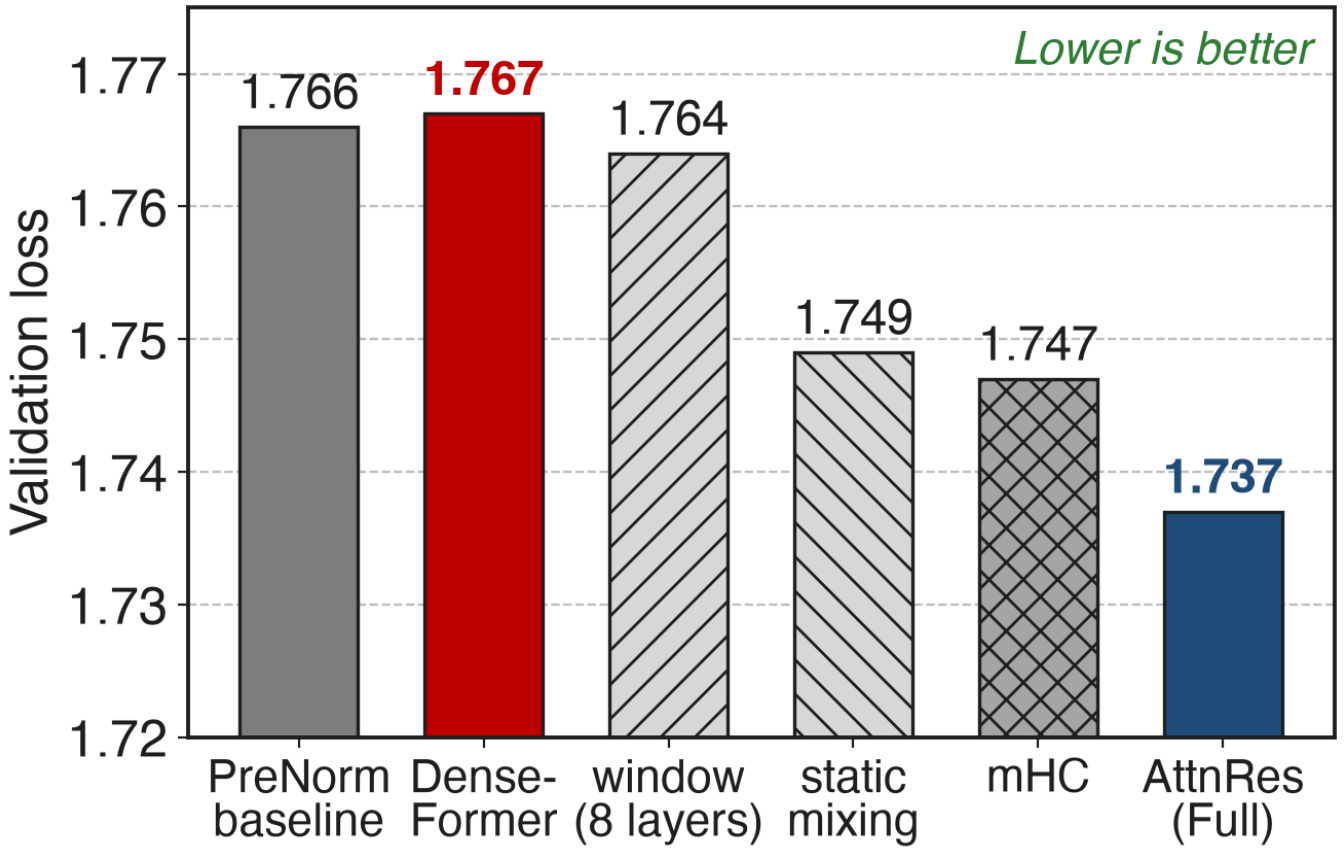
Standard residuals mix depth with fixed 1s

- Each layer sees **one** running sum
- No way to re-read an early layer
- PreNorm: magnitude grows **O(L)**, diluting layers



Static depth weights do not help

- DenseFormer: all layers, fixed weights
- mHC: input-dependent, but **34d** I/O per layer
- What helps: content-dependent selection



16-layer model, identical hyperparameters and compute (AttnRes paper, Table 4)

Insight: depth is a short sequence; attend over it

(a) standard residual

	emb	1	2	3	4	5	Σ
layer 1	1						1
layer 2	1	1					2
layer 3	1	1	1				3
layer 4	1	1	1	1			4
layer 5	1	1	1	1	1		5
layer 6	1	1	1	1	1	1	6

fixed 1s: depth-wise linear attention

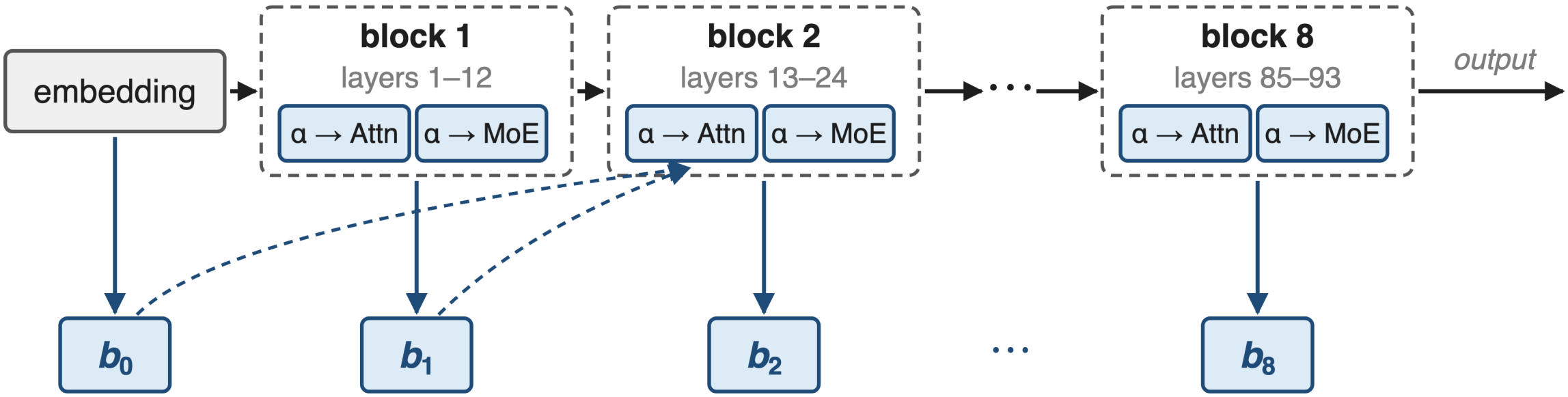
(b) AttnRes

	emb	1	2	3	4	5	Σ
layer 1	1						1
layer 2	.30	.70					1
layer 3	.15	.15	.70				1
layer 4	.40	.05	.05	.50			1
layer 5	.10	.55	.05	.05	.25		1
layer 6	.20	.05	.05	.05	.15	.50	1

softmax over sources, learned query

Residuals are depth-wise linear attention with weights 1; at L < 1000, softmax over depth is cheap

K3 attends over 9 depth sources per layer

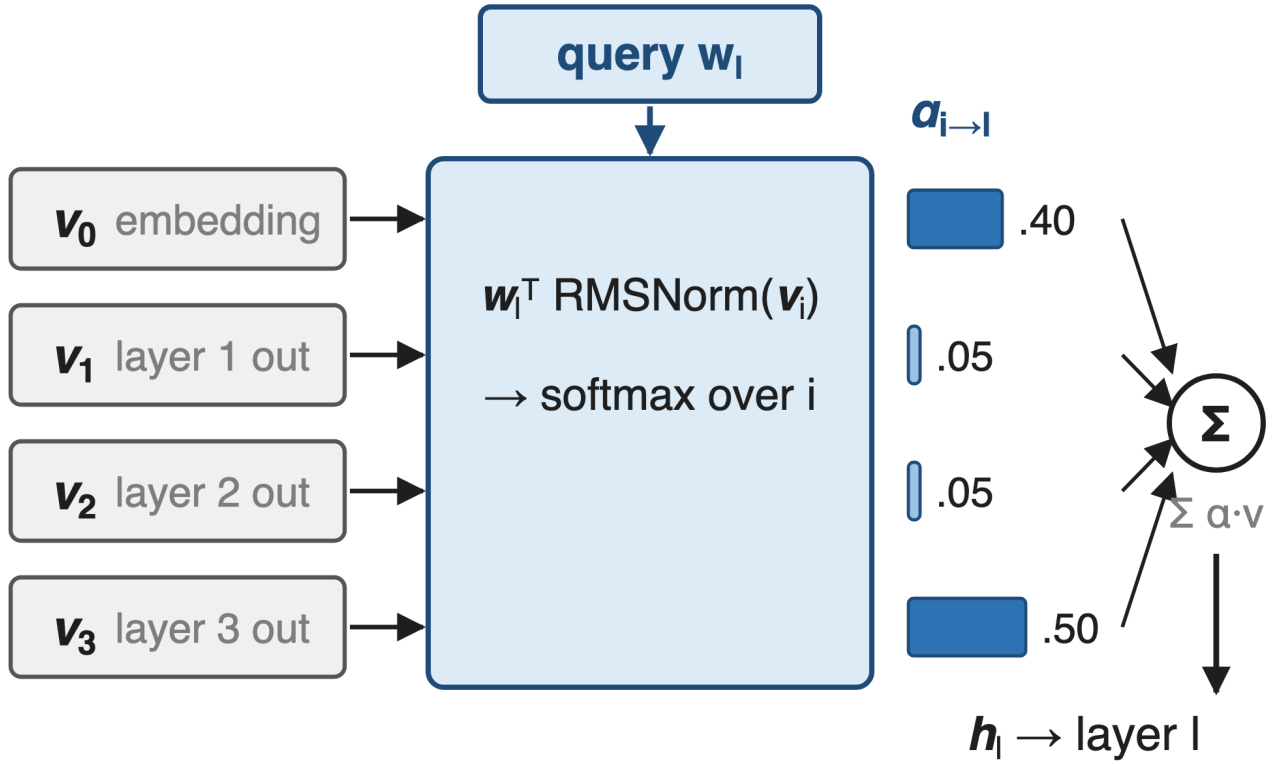


cached summaries b_0 … b_8: each α attends over the finished ones + its own block's partial sum

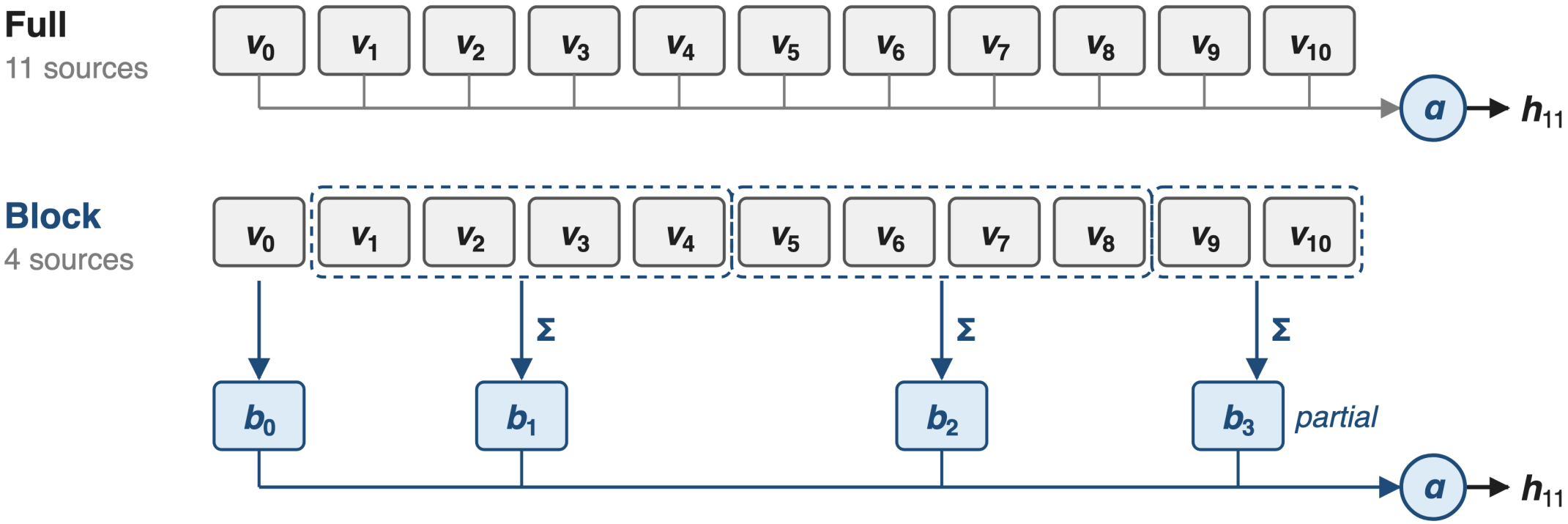
93 layers in 8 blocks of 12 (last block 9) plus the embedding · an AttnRes op before every attention and MoE sublayer, and one at the output

❶ One learned query per layer picks its sources

- Keys: RMSNorm of each source
- Query: one learned vector, zero-initialized
- Input: softmax-weighted sum of sources

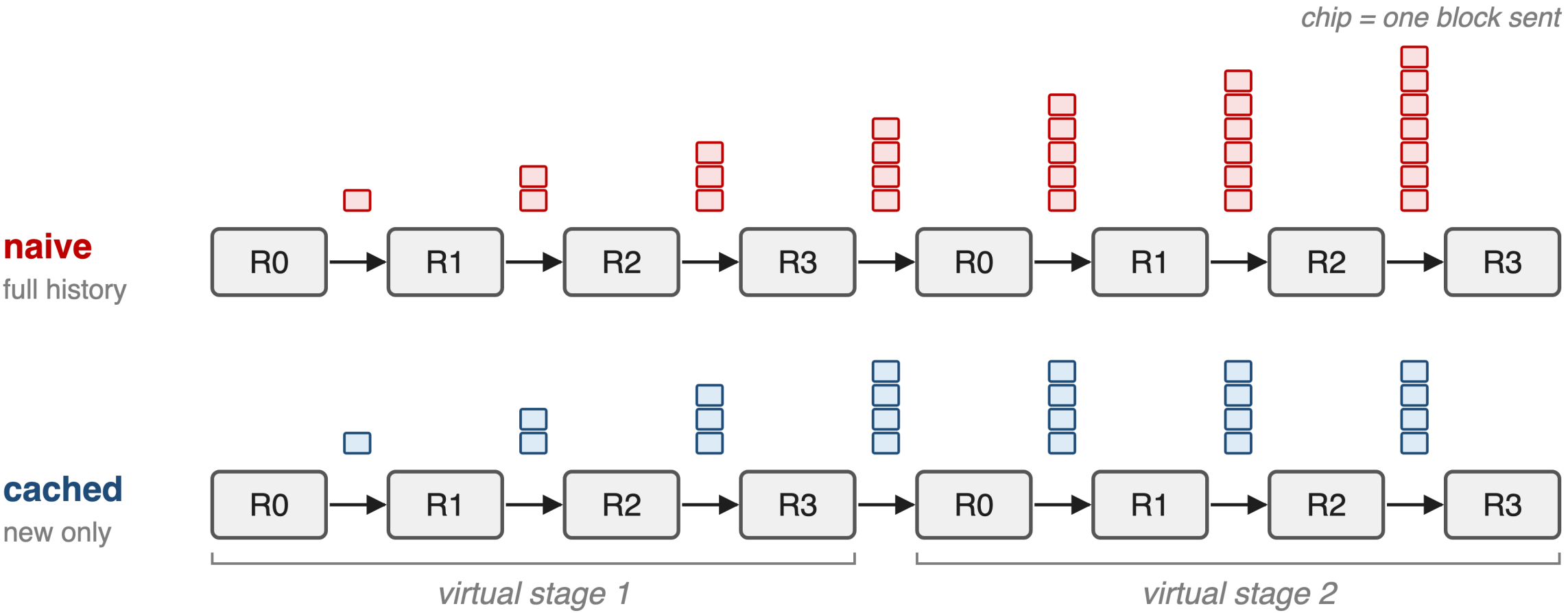


❷ Blocks cut depth state from L to N



About 8 blocks keep most of the gain: memory and pipeline traffic drop from O(Ld) to O(Nd)

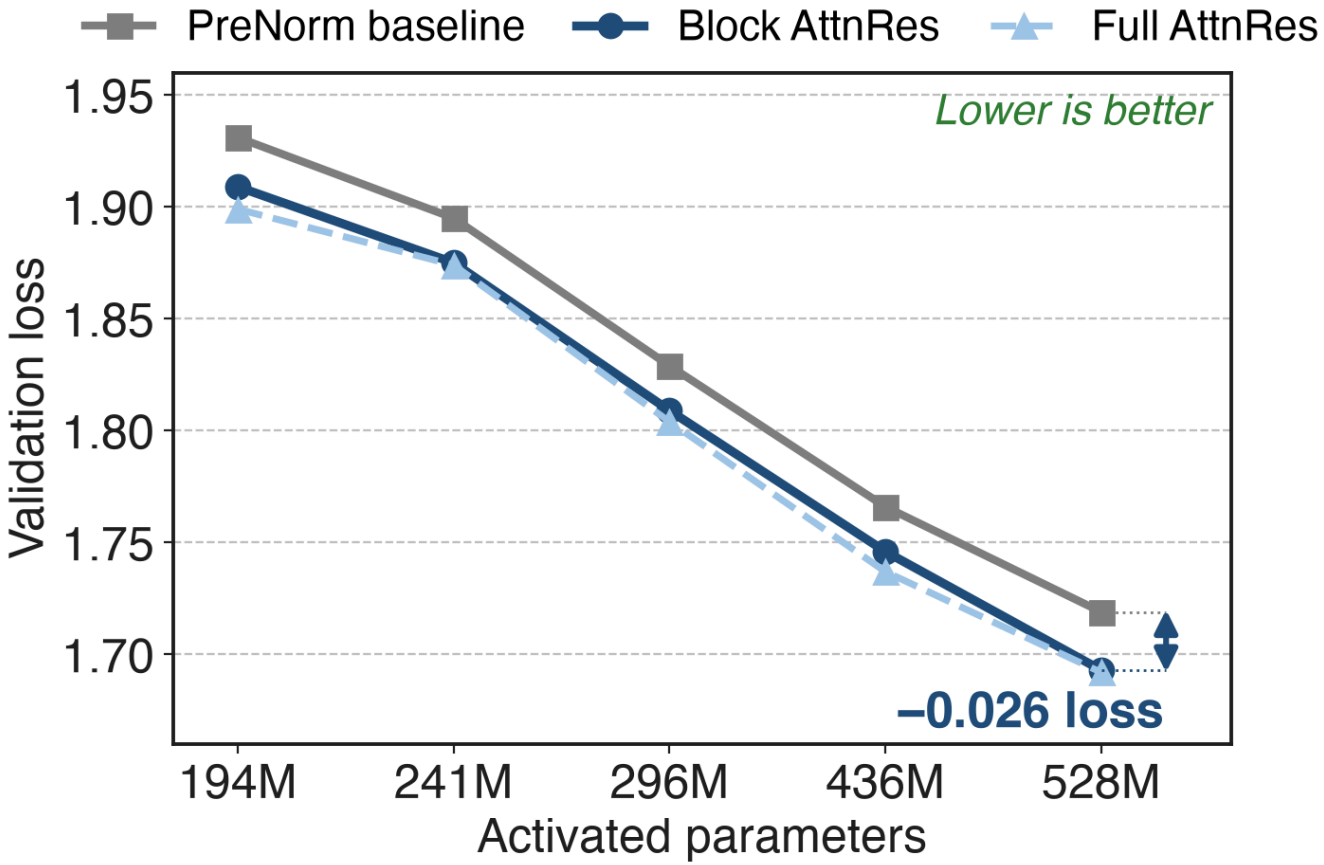
❸ Pipeline stages send only new blocks



Interleaved pipeline, P = 4 ranks, V = 2 virtual stages, one new block per chunk (schematic of AttnRes Eq. 7–8)

Block AttnRes lowers loss at all five sizes

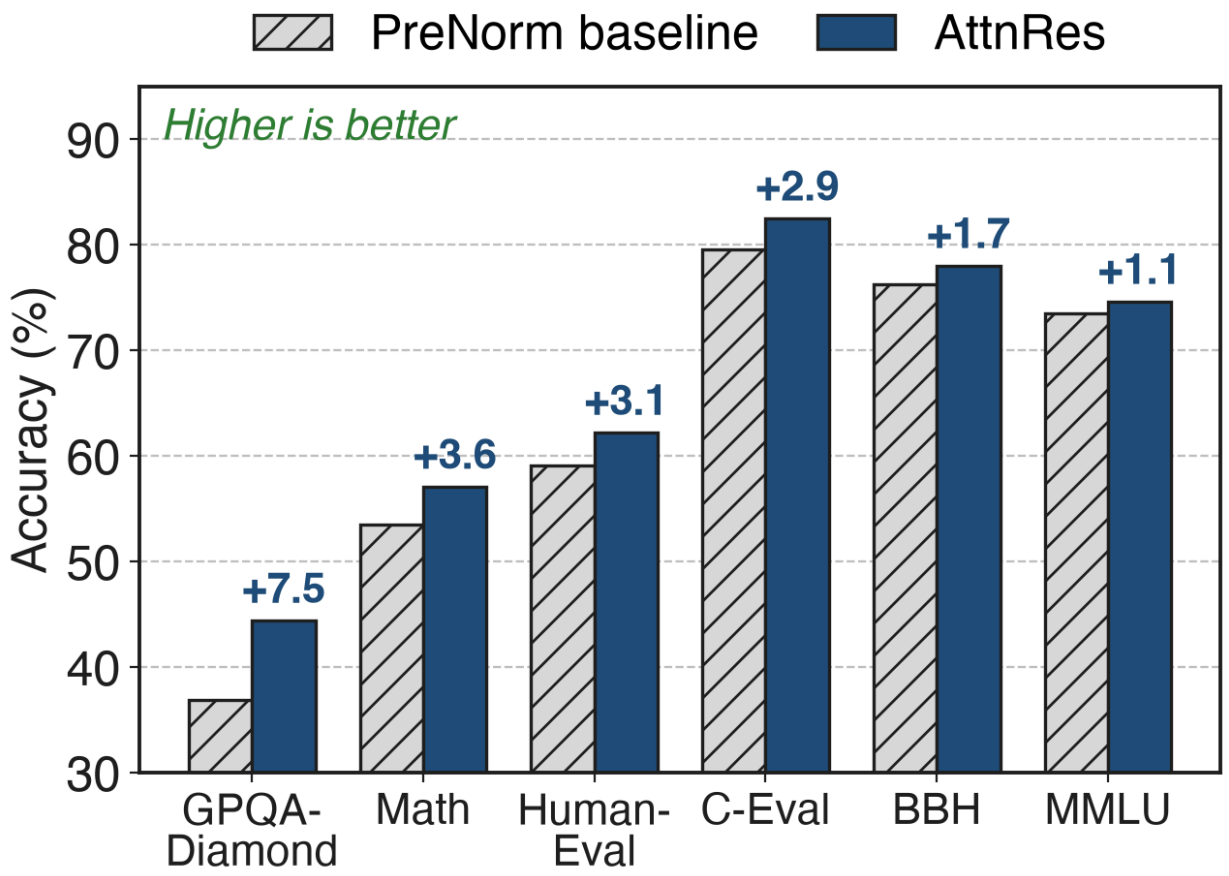
- **–0.020** to **–0.026** loss
- Fitted: baseline needs **1.25×** compute
- Full AttnRes only **0.001** ahead at 528M



Baseline-tuned hyperparameters, 8K context (AttnRes paper, Table 2)

+7.5 GPQA-Diamond on a 48B MoE

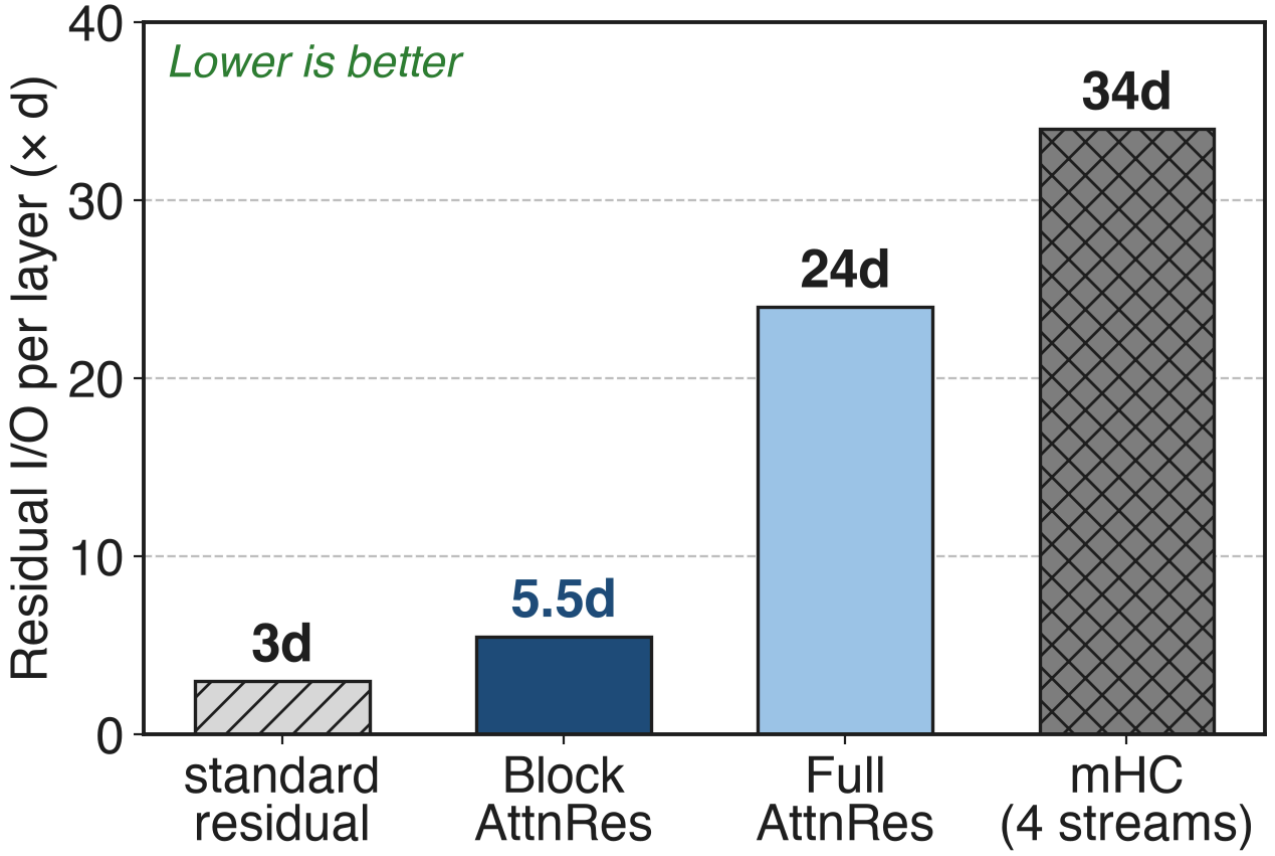
- Kimi Linear **48B** / **3B** active, **1.4T** tokens
- Matches or beats on all **15** benchmarks
- Biggest gains: multi-step reasoning, code



AttnRes paper, Table 3 (6 of 15 benchmarks shown)

Overhead: under 4% training, 2% inference

- Block AttnRes: **5.5d** per layer vs **3d**
- Training with pipelines: **< 4%** end-to-end
- Inference latency: **< 2%**



Typical L = 128, N = 8, S = 16; mHC with 4 streams (AttnRes paper, Table 1, §4)

The price: coarser blocks, pipeline plumbing

- Block **1.746** vs Full **1.737** loss
- Token-dependent query: **1.731**, costs d×d
- Needs block cache in PP + fused kernels
- Prefill cache: **15 GB** at 128K tokens
- Prefers deeper models: more latency
- **No K3-specific ablation** published

Summary

- Fixed residuals: **1.25×** compute for the same loss
- **Insight**: depth is short, so attend over it
- K3: **9** block sources per op, cached pipeline sends
- **+7.5** GPQA-Diamond; **< 4%** train, **< 2%** inference overhead

https://arxiv.org/abs/2603.15031